NEXT ENGINEER · _SHEERAT_

Design and Analysis of Algorithm

BCS503

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 5 | **Credits:** 4

L-T-P: 3-1-0 | **Scheme:** AKTU B.Tech 3rd year, 2024-25 onwards

UNIT 1	UNIT 2	UNIT 3	UNIT 4	UNIT 5
Introduction, Analysis and Sorting	Advanced Data Structures	Divide and Conquer and Greedy Methods	Dynamic Programming, Backtracking and Branch and Bound	Selected Topics

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: complexity tables, properties and algorithm steps.
3. **Solved examples**: the numericals that always appear (recurrences, sorting traces, knapsack, MST, shortest paths, Floyd, LCS, n-queens, string matching).
4. **Code / pseudocode to remember**: algorithms examiners ask you to write.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1. Asymptotic notations, master theorem with 4 recurrences, quick sort and heap sort traces, counting sort. Then Unit 2 properties (red-black, B-tree, binomial, Fibonacci heaps).
- **Afternoon (3 hours):** Units 3 and 4. Fractional knapsack, Kruskal and Prim, Dijkstra, Bellman-Ford, then 0/1 knapsack, Floyd-Warshall, matrix chain, LCS, n-queens and subset sum.
- **Evening (2 hours):** Unit 5. Rabin-Karp and KMP, P/NP/NP-complete, approximation and randomized algorithms. Then the "Last-hour checklist".
- **In the exam:** write the algorithm or recurrence, show the table or tree step by step, then state the time complexity. Examiners give marks for every step shown.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction, Analysis and Sorting

✍️ 2-mark definitions

- **Algorithm:** a finite sequence of well-defined steps that takes input and produces output. Properties: input, output, definiteness, finiteness and effectiveness.
- **Time complexity:** the number of basic operations as a function of input size n. **Space complexity:** the memory needed as a function of n.
- **Big-O (upper bound):** f(n) = O(g(n)) if there exist c > 0 and n0 such that 0 ≤ f(n) ≤ c·g(n) for all n ≥ n0.
- **Big-Omega (lower bound):** f(n) = Ω(g(n)) if f(n) ≥ c·g(n) ≥ 0 for all n ≥ n0.
- **Theta (tight bound):** f(n) = Θ(g(n)) if c1·g(n) ≤ f(n) ≤ c2·g(n) for all n ≥ n0.
- **Little-o and little-omega:** strict bounds; f = o(g) means f grows strictly slower than g (lim f/g = 0).
- **Best, worst and average case:** the minimum, maximum and expected running time over all inputs of size n.
- **Recurrence relation:** an equation that defines T(n) in terms of T on smaller inputs, e.g. T(n) = 2T(n/2) + n.
- **Stable sort:** a sort that keeps equal keys in their original relative order (merge, insertion, counting sort are stable; quick and heap sort are not).
- **In-place sort:** a sort that uses only O(1) extra memory (heap, quick, insertion, shell sort).
- **Comparison sort lower bound:** any comparison-based sort needs Ω(n log n) comparisons in the worst case (decision-tree argument: n! leaves need height ≥ $\log_2(n!)$).
- **Order statistic:** the i-th smallest element; the median is the middle one. It can be found in O(n) worst-case time (median of medians).

💡 Core concepts

Growth order: $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$.

Master theorem. For T(n) = aT(n/b) + f(n) with a ≥ 1, b > 1, compare f(n) with n^(log_b a):

1. If f(n) = O(n^(log_b a − ε)), then T(n) = Θ(n^(log_b a)).
2. If f(n) = Θ(n^(log_b a)), then T(n) = Θ(n^(log_b a) log n).
3. If f(n) = Ω(n^(log_b a + ε)) and a·f(n/b) ≤ c·f(n) for some c < 1, then T(n) = Θ(f(n)).

Other methods: substitution (guess and prove by induction), recursion tree (add the cost of each level), and iteration.

Sorting comparison

Algorithm	Best	Average	Worst	Space	Stable
Insertion	O(n)	O(n²)	O(n²)	O(1)	Yes
Shell	O(n log n)	depends on gaps (about n^1.25)	O(n²)	O(1)	No

Algorithm	Best	Average	Worst	Space	Stable
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting	O(n + k)	O(n + k)	O(n + k)	O(n + k)	Yes
Radix	O(d(n + k))	O(d(n + k))	O(d(n + k))	O(n + k)	Yes
Bucket	O(n + k)	O(n)	$O(n^2)$	O(n)	Yes

Shell sort: insertion sort on elements a gap apart, with the gap reduced (e.g. n/2, n/4, ..., 1). The last pass is ordinary insertion sort on an almost-sorted array.

Quick sort worst case happens when the pivot is always the smallest or largest (e.g. already sorted input with last-element pivot): $T(n) = T(n - 1) + n = \Theta(n^2)$. Randomized pivot gives expected $\Theta(n \log n)$.

Heap sort: build a max-heap in O(n), then repeatedly swap the root with the last element, reduce the heap size and heapify the root (O(log n) each).

🧮 Solved examples

Example 1 (master theorem).

- $T(n) = 2T(n/2) + n$: $n^{(\log_2 2)} = n = f(n)$, case 2, **$\Theta(n \log n)$**.
- $T(n) = 9T(n/3) + n$: $n^{(\log_3 9)} = n^2$, $f(n) = n$ is smaller, case 1, **$\Theta(n^2)$**.
- $T(n) = T(2n/3) + 1$: $a = 1$, $b = 3/2$, $n^0 = 1 = f(n)$, case 2, **$\Theta(\log n)$**.
- $T(n) = 3T(n/4) + n \log n$: $n^{(\log_4 3)} \approx n^{0.79}$, $f(n)$ is larger and $3(n/4)\log(n/4) \le (3/4)\, n \log n$, case 3, **$\Theta(n \log n)$**.

Example 2 (recursion tree for $T(n) = 2T(n/2) + cn$). Each level costs cn. There are $\log_2 n + 1$ levels. Total = $cn(\log_2 n + 1) = \Theta(n \log n)$.

Example 3 (quick sort partition, last element as pivot). A = [2, 8, 7, 1, 3, 5, 6, 4], pivot 4. Elements ≤ 4 move to the left: after the pass, A = [2, 1, 3, 4, 7, 5, 6, 8] with 4 at index 3 (0-based). Recurse on [2, 1, 3] and [7, 5, 6, 8].

Example 4 (counting sort). A = [2, 5, 3, 0, 2, 3, 0, 3], k = 5.

- Counts C[0..5] = [2, 0, 2, 3, 0, 1].
- Cumulative C = [2, 2, 4, 7, 7, 8].
- Placing from right to left gives B = [0, 0, 2, 2, 3, 3, 3, 5].

Example 5 (build max-heap). A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]. Heapify from index n/2 down to 1. Result: [16, 14, 10, 8, 7, 9, 3, 2, 4, 1].

NEXT ENGINEER sheerat

⌨️ Code to remember

```
PARTITION(A, p, r)                    QUICKSORT(A, p, r)
  x = A[r]; i = p - 1                   if p < r
  for j = p to r - 1                       q = PARTITION(A, p, r)
     if A[j] <= x                          QUICKSORT(A, p, q - 1)
        i = i + 1; swap A[i], A[j]         QUICKSORT(A, q + 1, r)
  swap A[i + 1], A[r]
  return i + 1

MAX-HEAPIFY(A, i)                     HEAPSORT(A)
  l = 2i; r = 2i + 1; largest = i        BUILD-MAX-HEAP(A)    // for i = n/2 downto 1: MAX-
HEAPIFY(A, i)
  if l <= heap-size and A[l] > A[i]      for i = n downto 2
     largest = l                           swap A[1], A[i]
  if r <= heap-size and A[r] > A[largest]   heap-size = heap-size - 1
     largest = r                           MAX-HEAPIFY(A, 1)
  if largest != i
     swap A[i], A[largest]; MAX-HEAPIFY(A, largest)
```

⭐ Important questions

1. Define asymptotic notations with graphs. State and apply the master theorem to given recurrences.
2. Solve recurrences by substitution and recursion tree methods.
3. Explain quick sort; analyse its best, average and worst case; trace it on given data.
4. Explain heap sort with BUILD-MAX-HEAP and trace it; explain shell sort and merge sort.
5. Prove the Ω(n log n) lower bound for comparison sorts; explain counting, radix and bucket sort; explain order statistics.

UNIT 2 Advanced Data Structures

✍️ 2-mark definitions

- **Red-black tree:** a self-balancing binary search tree where each node is red or black and the colouring rules keep the height at most $2 \log_2(n + 1)$.
- **Black height bh(x):** the number of black nodes on any path from x down to a leaf, not counting x.
- **Rotation:** an O(1) local restructuring (left or right) that keeps the BST order and is used to rebalance trees.
- **B-tree of minimum degree t:** a balanced multiway search tree where every node except the root has between t − 1 and 2t − 1 keys, and all leaves are at the same depth. Used for disks and databases.
- **Binomial tree Bk:** defined recursively; B0 is one node, and Bk is two Bk−1 trees linked so one root becomes the leftmost child of the other.
- **Binomial heap:** a collection of binomial trees, each heap-ordered, with at most one tree of each degree.
- **Fibonacci heap:** a collection of heap-ordered trees with lazy merging that gives very fast amortized operations; used in fast Dijkstra and Prim.
- **Trie (prefix tree):** a tree for storing strings where each edge is a character and each path from the root spells a prefix; search takes O(length of key).
- **Skip list:** a layered set of sorted linked lists where each higher level skips over more elements; search, insert and delete take expected O(log n).
- **Amortized analysis:** the average cost per operation over a worst-case sequence of operations (aggregate, accounting and potential methods).

💡 Core concepts

Red-black tree properties

1. Every node is red or black.
2. The root is black.
3. Every leaf (NIL) is black.
4. A red node has two black children (no two reds in a row).
5. Every path from a node to its descendant leaves has the same number of black nodes.

RB insertion: insert as in a BST and colour the new node z red. Fix-up while z's parent is red:

- Case 1, uncle red: recolour parent and uncle black, grandparent red, move z to the grandparent.
- Case 2, uncle black and z is an inner child: rotate at the parent to make it an outer child (becomes case 3).
- Case 3, uncle black and z is an outer child: rotate at the grandparent and swap colours of parent and grandparent.
- Finally colour the root black.

RB deletion: if the removed node was black, an "extra black" must be pushed up or removed using four cases based on the sibling w: w red; w black with two black children; w black with near child red and far child black; w black with far child red.

B-tree facts

- Height h ≤ log_t((n + 1)/2).
- Insert: always into a leaf. Split a full node (2t − 1 keys) on the way down: the median moves up to the parent.
- Delete: ensure every node visited has at least t keys, by borrowing from a sibling or merging with a sibling.

Binomial tree Bk properties: 2^k nodes, height k, exactly C(k, i) nodes at depth i, and the root has degree k with children Bk−1, ..., B0. A binomial heap with n nodes has at most ⌊log n⌋ + 1 trees, matching the 1 bits of n.

Heap operation costs

Operation	Binary heap	Binomial heap	Fibonacci heap (amortized)
Make heap	O(1)	O(1)	O(1)
Insert	O(log n)	O(log n)	O(1)
Minimum	O(1)	O(log n)	O(1)
Extract-min	O(log n)	O(log n)	O(log n)
Union	O(n)	O(log n)	O(1)
Decrease-key	O(log n)	O(log n)	O(1)
Delete	O(log n)	O(log n)	O(log n)

Binomial heap union: merge the root lists by degree, then link trees of equal degree (the larger key becomes a child of the smaller), like adding binary numbers with carries.

Fibonacci heap: insert adds a node to the root list; extract-min removes the min, adds its children to the root list, then consolidates trees of equal degree. Decrease-key cuts the node to the root list; cascading cut moves up marked parents. Maximum degree is O(log n), bounded via Fibonacci numbers.

🧮 Solved examples

Example 1 (RB insertion). Insert 10, 20, 30 into an empty RB tree.

- 10 becomes the black root.
- 20 is red, right child of 10. No violation.
- 30 is red, right child of 20 (red parent). Uncle (left of 10) is black NIL, and 30 is an outer child: case 3. Rotate left at 10 and swap colours: 20 is the black root with red children 10 and 30.

Example 2 (B-tree insertion, t = 2, so max 3 keys). Insert 10, 20, 30, 40, 50.

- 10, 20, 30 fill the root [10 20 30].
- Inserting 40: the root is full, so split it: 20 moves up. Root [20], children [10] and [30], then 40 goes to [30 40].

- Inserting 50: goes to [30 40 50].

Example 3 (binomial heap shape). A heap with n = 13 nodes (binary 1101) contains trees B3, B2 and B0 (8 + 4 + 1).

Example 4 (trie). Store "cat", "car", "dog": root has children c and d; c → a, and a has children t and r (both marking word ends); d → o → g. Searching "ca" reaches a non-end node, so "ca" is only a prefix.

Key points to remember

- An RB tree with n internal nodes has height at most $2 \log_2(n + 1)$.
- At most 2 rotations for an RB insertion and at most 3 for a deletion.
- B-trees grow at the root, not at the leaves.
- Fibonacci heaps make Dijkstra run in O(E + V log V).

⭐ Important questions

1. State the properties of red-black trees; prove the height bound; insert and delete a sequence of keys.
2. Define a B-tree; insert and delete keys in a B-tree of given order showing each split or merge.
3. Explain binomial trees and binomial heaps: properties, union and extract-min.
4. Explain Fibonacci heaps: structure, operations and amortized costs.
5. Explain tries and skip lists with examples.

UNIT 3 Divide and Conquer and Greedy Methods

✍️ 2-mark definitions

- **Divide and conquer:** split the problem into smaller sub-problems of the same type, solve them recursively, and combine the answers.
- **Greedy method:** build a solution step by step, always choosing the locally best option, and never reconsidering it.
- **Greedy-choice property:** a globally optimal solution can be reached by making a locally optimal choice.
- **Optimal substructure:** an optimal solution contains optimal solutions to its sub-problems.
- **Convex hull:** the smallest convex polygon that contains all the given points.
- **Minimum spanning tree (MST):** a spanning tree of a connected weighted graph with the minimum total edge weight.
- **Single-source shortest path:** the shortest distance from one source vertex to every other vertex.
- **Relaxation:** if d[u] + w(u, v) < d[v], then set d[v] = d[u] + w(u, v) and parent[v] = u.
- **Fractional knapsack:** items may be broken; solved by greedy on value/weight ratio.
- **Reliability design (optimal reliability allocation):** choosing how many copies of each component to use in series stages so that overall reliability is highest within a cost limit.

💡 Core concepts

Divide and conquer examples

Problem	Recurrence	Time
Binary search	T(n) = T(n/2) + 1	O(log n)
Merge sort	T(n) = 2T(n/2) + n	O(n log n)
Max-min (pairs)	T(n) = 2T(n/2) + 2	3n/2 − 2 comparisons
Ordinary matrix multiplication (D&C)	T(n) = 8T(n/2) + n^2	O(n^3)
Strassen's matrix multiplication	T(n) = 7T(n/2) + n^2	O(n^2.81)
Convex hull (divide and conquer)	T(n) = 2T(n/2) + n	O(n log n)

Strassen's method uses 7 products instead of 8:

- P = (A11 + A22)(B11 + B22), Q = (A21 + A22)B11, R = A11(B12 − B22), S = A22(B21 − B11), T = (A11 + A12)B22, U = (A21 − A11)(B11 + B12), V = (A12 − A22)(B21 + B22).
- C11 = P + S − T + V, C12 = R + T, C21 = Q + S, C22 = P + R − Q + U.

Convex hull algorithms: Graham scan (sort by polar angle, keep left turns using a stack; O(n log n)), Jarvis march / gift wrapping (O(nh) where h is the number of hull points), quickhull and divide-and-conquer merging of two hulls.

Greedy algorithms

Problem	Greedy rule	Time
Fractional knapsack	highest value/weight first	O(n log n)
Activity selection	earliest finish time first	O(n log n)
Huffman coding	merge two lowest frequencies	O(n log n)
Kruskal's MST	lightest edge that does not form a cycle (disjoint sets)	O(E log E)
Prim's MST	lightest edge leaving the tree (priority queue)	O(E log V)
Dijkstra	closest unvisited vertex	O((V + E) log V)

Kruskal vs Prim: Kruskal grows a forest and suits sparse graphs; Prim grows one tree from a start vertex and suits dense graphs.

Dijkstra vs Bellman-Ford

Point	Dijkstra	Bellman-Ford
Method	greedy	relaxes all edges V − 1 times (dynamic-programming style)
Negative edges	not allowed	allowed
Negative cycle	cannot detect	detects (an edge still relaxes in pass V)
Time	O((V + E) log V)	O(VE)

🧮 Solved examples

Example 1 (fractional knapsack). W = 50; items (weight, value): (10, 60), (20, 100), (30, 120). Ratios: 6, 5, 4.

- Take item 1 fully: weight 10, value 60.
- Take item 2 fully: weight 30, value 160.
- Remaining capacity 20: take 20/30 of item 3: value 80.
- Maximum value = **240**.

Example 2 (Kruskal's MST). Edges: B–C 1, A–B 2, A–C 3, B–D 4, C–D 5, C–E 6, D–E 7.

- Pick B–C (1), A–B (2). A–C (3) forms a cycle, skip. Pick B–D (4). C–D (5) forms a cycle, skip. Pick C–E (6). Four edges for five vertices: stop.
- MST = {B–C, A–B, B–D, C–E}, total weight **13**.

Prim from A gives the same tree: A–B (2), B–C (1), B–D (4), C–E (6).

Example 3 (Dijkstra). Directed edges: A→B 10, A→C 3, C→B 4, B→D 2, C→D 8, C→E 2, D→E 7. Source A.

Step	Picked	A	B	C	D	E
0	A	0	10	3	∞	∞
1	C	0	7	3	11	5
2	E	0	7	3	11	5

Step	Picked	A	B	C	D	E
3	B	0	7	3	9	5
4	D	0	7	3	9	5

Shortest distances: B 7 (A→C→B), C 3, D 9 (A→C→B→D), E 5 (A→C→E).

Example 4 (Huffman coding). Frequencies a 5, b 9, c 12, d 13, e 16, f 45.

- Merge 5 + 9 = 14; merge 12 + 13 = 25; merge 14 + 16 = 30; merge 25 + 30 = 55; merge 45 + 55 = 100.
- Codes: f = 0, c = 100, d = 101, a = 1100, b = 1101, e = 111.
- Total bits = 45·1 + 12·3 + 13·3 + 5·4 + 9·4 + 16·3 = 224.

⌨️ Code to remember

```
DIJKSTRA(G, w, s)
  for each v: d[v] = ∞; π[v] = NIL
  d[s] = 0; Q = all vertices (min-PQ on d)
  while Q not empty
     u = EXTRACT-MIN(Q)
     for each v adjacent to u
        if d[u] + w(u, v) < d[v]
           d[v] = d[u] + w(u, v); π[v] = u

BELLMAN-FORD(G, w, s)
  initialize d; d[s] = 0
  repeat |V| - 1 times: for each edge (u, v): RELAX(u, v)
  for each edge (u, v): if d[u] + w(u, v) < d[v] return FALSE   // negative cycle
  return TRUE
```

```
KRUSKAL(G, w)
   A = ∅
   for each v: MAKE-SET(v)
   sort edges by weight
   for each edge (u, v) in order
      if FIND-SET(u) != FIND-SET(v)
         A = A U {(u, v)}; UNION(u, v)
   return A
```

⭐ Important questions

1. Explain divide and conquer; derive the complexity of Strassen's matrix multiplication.
2. Explain convex hull algorithms (Graham scan) with an example.
3. Solve a fractional knapsack instance; prove the greedy choice is optimal.
4. Find the MST of a given graph using Prim's and Kruskal's algorithms; compare them.
5. Apply Dijkstra's and Bellman-Ford algorithms to a given graph; explain negative-cycle detection.

UNIT 4 Dynamic Programming, Backtracking and Branch and Bound

✍️ 2-mark definitions

- **Dynamic programming (DP):** solving a problem by combining solutions to overlapping sub-problems, storing each result in a table so it is computed only once.
- **Overlapping sub-problems:** the same smaller problems are solved many times by plain recursion (e.g. Fibonacci).
- **Memoization vs tabulation:** memoization is top-down recursion with a cache; tabulation is bottom-up filling of a table.
- **0/1 knapsack:** each item is taken fully or not at all; solved by DP in O(nW) time.
- **All-pairs shortest path:** shortest distances between every pair of vertices (Floyd-Warshall).
- **Transitive closure (Warshall):** a matrix whose (i, j) entry is 1 if j is reachable from i.
- **Backtracking:** building a solution one component at a time and abandoning (backtracking) a partial solution as soon as it cannot lead to a valid answer. It explores a state-space tree depth-first.
- **Branch and bound:** exploring the state-space tree using bounds; a node is pruned if its bound shows it cannot beat the best solution found so far. Usually breadth-first or least-cost first.
- **Bounding function:** a function that estimates the best possible value obtainable from a node.
- **Hamiltonian cycle:** a cycle that visits every vertex exactly once and returns to the start.
- **Graph colouring (m-colouring):** assign at most m colours to vertices so that adjacent vertices differ; the smallest such m is the chromatic number.

💡 Core concepts

DP vs greedy vs divide and conquer

Point	Divide and conquer	Dynamic programming	Greedy
Sub-problems	independent	overlapping	one choice at a time
Storage	none	table	none
Optimality	exact	always optimal (with optimal substructure)	optimal only when greedy choice holds
Example	merge sort	0/1 knapsack	fractional knapsack

Backtracking vs branch and bound

Point	Backtracking	Branch and bound
Problem type	decision / find all solutions	optimization

Point	Backtracking	Branch and bound
Search	depth-first	BFS or least-cost (best-first)
Pruning	constraint violated	bound worse than best so far
Examples	n-queens, subset sum, graph colouring, Hamiltonian	TSP, 0/1 knapsack, job assignment

DP recurrences to remember

- 0/1 knapsack: K[i][w] = max(K[i−1][w], v_i + K[i−1][w − w_i]) if $w_i \le w$, else K[i−1][w].
- Floyd-Warshall: D^k[i][j] = min(D^(k−1)[i][j], D^(k−1)[i][k] + D^(k−1)[k][j]). Time $O(V^3)$.
- Warshall: T^k[i][j] = T^(k−1)[i][j] OR (T^(k−1)[i][k] AND T^(k−1)[k][j]).
- Matrix chain: m[i][j] = min over $i \le k < j$ of m[i][k] + m[k+1][j] + p(i−1)·p(k)·p(j). Time $O(n^3)$.
- LCS: c[i][j] = c[i−1][j−1] + 1 if $x_i = y_j$, else max(c[i−1][j], c[i][j−1]). Time O(mn).
- Resource allocation: f(i, x) = max over $k \le x$ of (profit of giving k units to project i + f(i − 1, x − k)).

TSP by branch and bound (reduced-cost matrix method).

1. Reduce every row and column so each has a 0; the total subtracted is the lower bound of the root.
2. For an edge (i, j), set row i and column j to ∞ and (j, 1) to ∞, reduce again; cost = parent cost + M[i][j] + new reduction.
3. Expand the live node with the smallest cost; stop when a complete tour has cost no greater than any live node.

🧮 Solved examples

Example 1 (0/1 knapsack). W = 50; items (10, 60), (20, 100), (30, 120). Options: items 1+2 = 160 (weight 30), items 1+3 = 180 (weight 40), items 2+3 = 220 (weight 50), all three do not fit. Maximum = **220** with items 2 and 3. (Greedy by ratio would give 160, which shows greedy fails for 0/1 knapsack.)

Example 2 (0/1 knapsack table). W = 5; items (w, v): (2, 3), (3, 4), (4, 5), (5, 6).

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1 (2, 3)	0	0	3	3	3	3
2 (3, 4)	0	0	3	4	4	7
3 (4, 5)	0	0	3	4	5	7
4 (5, 6)	0	0	3	4	5	7

Maximum value **7** with items 1 and 2.

Example 3 (Floyd-Warshall). Edges 1→2 = 4, 1→3 = 11, 2→1 = 6, 2→3 = 2, 3→1 = 3.

- D0 = [0 4 11; 6 0 2; 3 ∞ 0]
- k = 1: D[3][2] = 3 + 4 = 7. D1 = [0 4 11; 6 0 2; 3 7 0]
- k = 2: D[1][3] = 4 + 2 = 6. D2 = [0 4 6; 6 0 2; 3 7 0]
- k = 3: D[2][1] = 2 + 3 = 5. D3 = **[0 4 6; 5 0 2; 3 7 0]**

Example 4 (matrix chain). A1 10×30, A2 30×5, A3 5×60.

- (A1A2)A3 = 10·30·5 + 10·5·60 = 1500 + 3000 = 4500.
- A1(A2A3) = 30·5·60 + 10·30·60 = 9000 + 18000 = 27000.
- Best: **(A1A2)A3 with 4500** multiplications.

Example 5 (LCS). X = ABCBDAB, Y = BDCABA. The LCS length is **4**, e.g. BCBA (also BDAB, BCAB).

Example 6 (4-queens by backtracking). Place queen 1 in column 1; queen 2 can go in column 3 or 4; every branch from column 1 fails. Backtrack to column 2: queens at (1, 2), (2, 4), (3, 1), (4, 3). Solutions: **(2, 4, 1, 3)** and its mirror **(3, 1, 4, 2)**.

Example 7 (sum of subsets). S = {5, 10, 12, 13, 15, 18}, M = 30. Solutions: **{5, 10, 15}, {5, 12, 13}, {12, 18}**. Prune a node when the current sum plus the next element exceeds M, or when the current sum plus all remaining elements is less than M.

Example 8 (graph colouring). A cycle of 4 vertices can be coloured with 2 colours (alternate); a triangle needs 3. Backtracking tries colours 1 ... m for each vertex in order and backtracks when a neighbour already has that colour.

⌨️ Code to remember

```
N-QUEENS(k, n)                          PLACE(k, i)   // can queen k go in column i?
  for i = 1 to n                          for j = 1 to k - 1
     if PLACE(k, i)                          if x[j] == i or |x[j] - i| == |j - k|
        x[k] = i                                return false
        if k == n: print x[1..n]           return true
        else N-QUEENS(k + 1, n)

FLOYD-WARSHALL(W)
  D = W
  for k = 1 to n
     for i = 1 to n
        for j = 1 to n
           D[i][j] = min(D[i][j], D[i][k] + D[k][j])
  return D
```

⭐ Important questions

1. Explain the elements of dynamic programming; solve a 0/1 knapsack instance with the DP table.
2. Explain Warshall's and Floyd's algorithms and apply Floyd's to a given graph.
3. Solve matrix chain multiplication / LCS / resource allocation problems using DP.
4. Explain backtracking; solve n-queens, sum of subsets, graph colouring and Hamiltonian cycle with state-space trees.
5. Explain branch and bound; solve a TSP instance with the reduced-cost matrix method.

UNIT 5 Selected Topics

✍️ 2-mark definitions

- **Algebraic computation:** algorithms for arithmetic on polynomials, matrices and large numbers, e.g. polynomial multiplication with FFT.
- **Fast Fourier Transform (FFT):** an O(n log n) divide-and-conquer method to evaluate a degree-(n−1) polynomial at the n complex n-th roots of unity (the DFT), and to interpolate back.
- **String matching:** finding all positions where a pattern P of length m occurs in a text T of length n.
- **Spurious hit:** in Rabin-Karp, a window whose hash matches the pattern's hash but whose characters do not match.
- **Prefix function π (KMP):** π[q] is the length of the longest proper prefix of P[1..q] that is also a suffix of it.
- **Class P:** decision problems solvable in polynomial time by a deterministic algorithm.
- **Class NP:** decision problems whose "yes" answers can be verified in polynomial time given a certificate (solvable by a nondeterministic machine in polynomial time).
- **NP-hard:** a problem to which every NP problem reduces in polynomial time; it need not be in NP.
- **NP-complete:** a problem that is both in NP and NP-hard, e.g. SAT, 3-SAT, clique, vertex cover, Hamiltonian cycle, subset sum.
- **Polynomial-time reduction (A ≤p B):** a polynomial-time transformation of instances of A into instances of B with the same answer.
- **Approximation algorithm:** a polynomial-time algorithm that gives a near-optimal solution with a guaranteed ratio ρ(n) to the optimum.
- **Randomized algorithm:** an algorithm that makes random choices. Las Vegas: always correct, running time random (randomized quick sort). Monte Carlo: fixed time, may be wrong with small probability (Miller-Rabin primality test).

💡 Core concepts

String matching algorithms

Algorithm	Preprocessing	Matching (worst)	Idea
Naive	none	O((n − m + 1)m)	try every shift
Rabin-Karp	O(m)	O((n − m + 1)m), expected O(n + m)	rolling hash mod q
Finite automaton	O(m·|Σ|)	O(n)	build a DFA for P
KMP	O(m)	O(n)	prefix function avoids re-checking

Rabin-Karp rolling hash: $t(s+1) = (d(t(s) - T[s+1]\cdot h) + T[s+m+1]) \bmod q$, where $h = d^{(m-1)} \bmod q$.

Relations among classes: P ⊆ NP. If any NP-complete problem is in P, then P = NP. Whether P = NP is unsolved. NP-complete ⊆ NP-hard.

Cook's theorem: SAT (Boolean satisfiability) is NP-complete. It was the first problem proved NP-complete.

Proving a problem X is NP-complete: (1) show X ∈ NP (a certificate can be checked in polynomial time); (2) reduce a known NP-complete problem to X in polynomial time. Common chain: SAT → 3-SAT → Clique → Vertex cover; 3-SAT → Subset sum; Hamiltonian cycle → TSP.

Approximation algorithms

- **Vertex cover (ratio 2):** pick any edge (u, v), add both u and v, remove all edges touching them, repeat. The result is at most twice the optimum.
- **TSP with triangle inequality (ratio 2):** build an MST, do a preorder walk, and skip repeated vertices.
- **Set cover (greedy):** ratio about ln n.

FFT key points. The n-th roots of unity are ω^k = e^(2πik/n). Split A(x) = A_even(x²) + x·A_odd(x²); the halving lemma means only n/2 distinct squares, giving T(n) = 2T(n/2) + O(n) = O(n log n). Polynomial multiplication: evaluate both with FFT, multiply point-wise, interpolate with inverse FFT, all in O(n log n) instead of O(n²).

🧮 Solved examples

Example 1 (Rabin-Karp). Text T = 2359023141526739921, pattern P = 31415, q = 13, d = 10.

- P mod 13 = 31415 mod 13 = **7**.
- Windows of length 5 with hash 7: 31415 (a valid match, at shift 6) and 67399 (67399 mod 13 = 7, a **spurious hit**, rejected after character comparison).

Example 2 (KMP prefix function). P = abababaca.

q	1	2	3	4	5	6	7
P[q]	a	b	a	b	a	c	a
π[q]	0	0	1	2	3	0	1

Example 3 (naive matching count). T = AAAAAAAAAA (n = 10), P = AAAAB (m = 5): every shift compares 5 characters, giving (10 − 5 + 1)·5 = 30 comparisons, which is the worst case.

Example 4 (vertex cover is in NP). Certificate: a set C of k vertices. Check |C| ≤ k and that every edge has at least one end in C: O(V + E). So vertex cover ∈ NP. Reduction from clique: G has a clique of size k if and only if the complement graph has a vertex cover of size |V| − k.

Example 5 (approximate vertex cover). Edges: a–b, b–c, c–d, c–e, d–e, d–f, d–g, e–f. Pick b–c: add b, c, remove their edges. Pick e–f: add e, f. Pick d–g: add d, g. Cover = {b, c, d, e, f, g} (size 6). An optimal cover is {b, d, e} (size 3), so the ratio is exactly 2 here.

⌨️ Code to remember

```
KMP-MATCHER(T, P)
  π = COMPUTE-PREFIX(P); q = 0
  for i = 1 to n
     while q > 0 and P[q+1] != T[i]
        q = π[q]
     if P[q+1] == T[i]: q = q + 1
     if q == m
        print "match at shift" i - m
        q = π[q]

RANDOMIZED-PARTITION(A, p, r)
  i = RANDOM(p, r); swap A[r], A[i]
  return PARTITION(A, p, r)
```

```
COMPUTE-PREFIX(P)
  π[1] = 0; k = 0
  for q = 2 to m
     while k > 0 and P[k+1] != P[q]
        k = π[k]
     if P[k+1] == P[q]: k = k + 1
     π[q] = k
  return π
```

⭐ Important questions

1. Explain the naive, Rabin-Karp and KMP string-matching algorithms with examples and complexities.
2. Explain FFT and its use in polynomial multiplication.
3. Define P, NP, NP-hard and NP-complete with a diagram; state Cook's theorem; prove that a given problem is NP-complete.
4. Explain approximation algorithms for vertex cover and TSP with their ratio bounds.
5. Explain randomized algorithms (Las Vegas vs Monte Carlo) with randomized quick sort.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ O, Ω, Θ definitions with c and n0; master theorem three cases.
- ☐ Sorting table (best/average/worst, stable, in place); comparison lower bound n log n.
- ☐ Quick sort partition; BUILD-MAX-HEAP; counting sort steps.
- ☐ RB tree 5 properties and 3 insertion cases; B-tree split; binomial tree Bk properties; heap cost table.
- ☐ Strassen 7 products; convex hull (Graham scan).
- ☐ Fractional knapsack 240 example; Huffman; Kruskal vs Prim.
- ☐ Dijkstra table; Bellman-Ford V − 1 passes and negative cycle check.
- ☐ 0/1 knapsack recurrence; Floyd D^k formula; matrix chain; LCS.
- ☐ n-queens PLACE condition; sum of subsets pruning; graph colouring; TSP reduced matrix.
- ☐ Rabin-Karp spurious hit; KMP prefix table.
- ☐ P, NP, NP-hard, NP-complete diagram; Cook's theorem; reduction steps.
- ☐ Vertex cover 2-approximation; Las Vegas vs Monte Carlo.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Introduction to Algorithms	Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein	PHI	WorldCat
2	Fundamentals of Computer Algorithms	Ellis Horowitz, Sartaj Sahni	Galgotia / Universities Press	WorldCat
3	The Design and Analysis of Computer Algorithms	A. V. Aho, J. E. Hopcroft, J. D. Ullman	Pearson Education	WorldCat
4	Design and Analysis of Algorithms	R. C. T. Lee et al.	McGraw Hill	WorldCat
5	Foundations of Algorithms	Richard E. Neapolitan	Jones & Bartlett Learning	WorldCat
6	Algorithm Design	Jon Kleinberg, Éva Tardos	Pearson	WorldCat
7	Algorithm Design: Foundations, Analysis, and Internet Examples	Michael T. Goodrich, Roberto Tamassia	Wiley	WorldCat
8	Data Structures and Their Algorithms	Harry R. Lewis, Larry Denenberg	HarperCollins	WorldCat
9	Algorithms (4th ed.)	Robert Sedgewick, Kevin Wayne	Addison Wesley	WorldCat
10	Algorithm Design and Analysis	Harsh Bhasin	Oxford University Press	WorldCat
11	Algorithmics: Theory and Practice	Gilles Brassard, Paul Bratley	Prentice Hall	WorldCat

Syllabus source: AKTU B.Tech 3rd year syllabus, 2024-25 (copy hosted by an affiliated college, PDF)